Week 15
Sequence Modelling for RL: Decision Transformers and Diffusion Policies

STUDY-RL — a self-paced Deep RL curriculum

May 21, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 5
- 2.1 Derivation of DT as a likelihood model . . . 5
- 2.2 Generation at deployment . . . 5
- 2.3 Trajectory Transformer . . . 6
- 2.4 Diffusion policies (overview) . . . 6
- 2.5 Q-Transformer . . . 6

3 Algorithms 7
- 3.1 Worked numerical example . . . 7

4 Implementation walkthrough 8

5 Empirical results 9

6 Exercises 11

7 Further reading 12

Learning objectives

Key Idea

After this week you will:

1. Recast offline RL as autoregressive sequence modelling: derive the Decision Transformer (DT) loss and explain why no Bellman backup is required.
2. Implement a tiny DT and verify *return-conditioned generation*: high target returns produce high empirical returns.
3. Sketch the Trajectory Transformer beam-search variant and contrast it with DT.
4. Derive the DDPM score-matching loss and connect it to diffusion policies (Chi 2023), Diffuser (Janner 2022), Diffusion-QL, and IDQL.
5. Summarise Q-Transformer and its place in offline-to-online robotic policies.
6. Translate DT-style "career trajectory" modelling into a concrete recruitment use case.

1 Motivation & intuition

The standard Bellman backup is just one way to extract a policy from a logged dataset. An alternative, far less obvious in retrospect, is to treat each logged trajectory as a *language model* would treat a sentence and predict the next token (here, the next action) conditioned on a target return.

It is the way that propagates value, which is dangerous on out-of-support actions — see week 14.

Intuition. Imagine you have logged 10 000 trajectories from your driving school. Each trajectory has a final grade (the return-to-go at $t = 0$). A language model that reads "$(R = \text{A+}), s_0, a_0, (R - r_0), s_1, a_1, \ldots$" learns to write trajectories ending with grade A+. At test time you prepend "$(R = \text{A+}), s_0$" and ask it to autoregress. That is Decision Transformer.

Common pitfall

DT does *not* solve the offline RL problem in general. It is an imitator conditioned on a desired outcome. If the desired outcome lies outside the convex hull of logged returns, you have asked for something the dataset cannot teach. The model will respond plausibly but inaccurately.

2 Mathematical formulation

Definition 2.0.1 (DT token sequence)**.** For a trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ of length T and discount factor $\gamma \in (0, 1]$, define the return-to-go

$$R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

The DT sequence is the interleaved tuple

$$X = (R_0, s_0, a_0, R_1, s_1, a_1, \ldots, R_{T-1}, s_{T-1}, a_{T-1}),$$

of length $3T$.

Definition 2.0.2 (DT objective)**.** For a transformer f_θ that produces, at the position of s_t, a distribution over actions, the DT loss is

$$\mathcal{L}_{\mathrm{DT}}(\theta) = -\mathbb{E}_{\tau \sim \mathcal{D}} \sum_{t=0}^{T-1} \log f_\theta(a_t \mid R_{\leq t}, s_{\leq t}, a_{<t}).$$

2.1 Derivation of DT as a likelihood model

Treat the trajectory as a sample from a joint distribution $p(R_0, s_0, a_0, \ldots)$. Apply the chain rule:

$$\log p(\tau) = \sum_{t=0}^{T-1} \big[\log p(R_t \mid X_{<t}) + \log p(s_t \mid X_{<t}, R_t) + \log p(a_t \mid X_{<t}, R_t, s_t)\big].$$

DT only fits the third term — the action conditional. The reward dynamics and the transition $p(s_t \mid \ldots)$ are left to the environment at deployment time. This is the crucial design choice: *we never model the world; we only model the data-generating distribution of actions.*

Theorem 2.1.1 (Conditional sufficiency of R_t)**.** *If the behaviour policy is a function only of the state s_t and an exogenous "intent" variable $z \sim p(z)$ that determines the realised return R_0, then conditioning on R_t recovers the optimal-given-intent action:*

$$p(a_t \mid s_t, R_t) = \mathbb{E}_{z \mid R_t, s_t}[\pi_\beta(a_t \mid s_t, z)].$$

Sketch. R_t is a sufficient statistic of z under the assumed factorisation; marginalising over the posterior $p(z \mid R_t, s_t)$ gives the displayed equation. This is the formal version of "conditioning on the desired outcome selects the right intent". □

2.2 Generation at deployment

At test time the agent specifies a target return R_0^{target}. After observing r_t at step t, the target return is updated as

$$R_{t+1}^{\mathrm{target}} = R_t^{\mathrm{target}} - r_t.$$

The model is greedy or sampled w.r.t. $f_\theta(a_t \mid R_{\leq t}, s_{\leq t}, a_{<t})$. This is a literal copy of language-model generation.

2.3 Trajectory Transformer

Janner et al. 2021 take the same idea further: discretise *state and reward* as well, and use beam search over (s, a, r) triples. Trajectories are sampled and scored by predicted return. DT trades planning for speed; TT trades speed for planning.

2.4 Diffusion policies (overview)

The DDPM forward process noises a sample $x_0 \sim q(x_0)$ along T steps with

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I),$$

with marginal $q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I)$ where $\bar{\alpha}_t = \prod_{k \le t}(1-\beta_k)$.

Theorem 2.4.1 (DDPM denoising loss)**.** *The reverse process $p_\theta(x_{t-1} \mid x_t)$ parametrised as Gaussian with mean predicted via a noise predictor $\varepsilon_\theta(x_t, t)$ has variational lower bound that simplifies, up to a constant, to*

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x_0, \varepsilon, t}\Big[\big\|\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon,\, t)\big\|_2^2\Big].$$

Sketch. The KL between $q(x_{t-1} \mid x_t, x_0)$ and $p_\theta(x_{t-1} \mid x_t)$ reduces to a weighted MSE on ε. Ho et al. 2020 then drop the weight. □

For **diffusion policies** (Chi 2023), x_0 is an action sequence $(a_t, a_{t+1}, \ldots, a_{t+H-1})$ conditioned on observation history; the noise predictor $\varepsilon_\theta(x_t, t, o)$ is conditioned on the observation. **Diffuser** (Janner 2022) generalises this to plan over (s, a) trajectories with an explicit value-function guidance term added to the noise-prediction at inference. **Diffusion-QL** (Wang 2022) adds a Q-maximising guidance term: $\nabla_a Q_\phi$ pushes the sampled action toward higher value. **IDQL** (Hansen-Estruch 2023) trains a value model in IQL style and uses it to re-rank diffusion samples.

2.5 Q-Transformer

Chebotar et al. 2023 quantise each action dimension into K bins and treat the per-dimension action choices as an autoregressive sequence over a single timestep. The transformer outputs a Q-value per bin per dimension; Bellman backup is performed over the per-dimension max. This unifies CQL-style conservatism (via the bin softmax) with transformer scaling.

3 Algorithms

Algorithm 1: Decision Transformer (training)

Input: Dataset of trajectories $\mathcal{D}$, context length K, transformer f_θ.
for *step = 1, ..., N* **do**
  Sample a trajectory $\tau \sim \mathcal{D}$ and a window $[t, t+K-1]$;
  Compute returns-to-go $R_t, \ldots, R_{t+K-1}$;
  Build the interleaved sequence X;
  Forward $f_\theta(X)$, compute cross-entropy loss only at action positions;
  Backprop, AdamW step;

Complexity per step: $O(K^2 d)$ for self-attention, $O(Kd^2)$ for the FFN, d = hidden width. On our toy domain $K = 8$, $d = 64$: trivial.

3.1 Worked numerical example

A length-3 trajectory with rewards $(-1, -1, +10)$ has returns-to-go $(8, 9, 10)$. The DT sees $(8, s_0, a_0, 9, s_1, a_1, 10, s_2, a_2)$ and learns to predict a_0 given $(8, s_0)$, a_1 given $(8, s_0, a_0, 9, s_1)$, and a_2 given the full prefix. At test time the agent specifies target $R_0 = 10$, observes $r_0 = -1$, decrements the next target to 9, and so on. A target of 20 would be *out of distribution*: the model can extrapolate the conditioning but the resulting actions are not guaranteed to be useful.

4 Implementation walkthrough

The transformer interleaves three token streams (R, s, a) along the time dimension, with a single learnt positional embedding indexed by "timestep" rather than "token index" (this is the trick of DT — it ensures returns / states / actions of the same time share a position).

```
tokens = torch.stack([rt, st, at], dim=2).reshape(B, 3*K, hidden)
mask = torch.triu(torch.ones(3*K, 3*K, dtype=bool), diagonal=1)
h = transformer(tokens, mask=mask, is_causal=True)
logits = action_head(h[:, 1::3, :])  # pick state positions
```

At deployment, returns-to-go are *user-specified* and decremented by observed rewards.

5 Empirical results

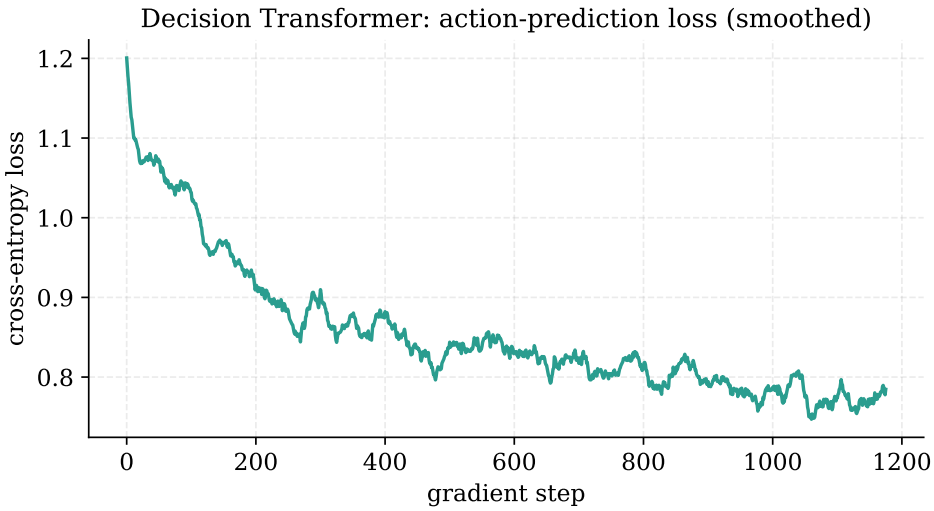


Figure 5.1: Decision Transformer action-prediction loss (smoothed) over 1200 gradient steps on the synthetic GridWorld dataset.

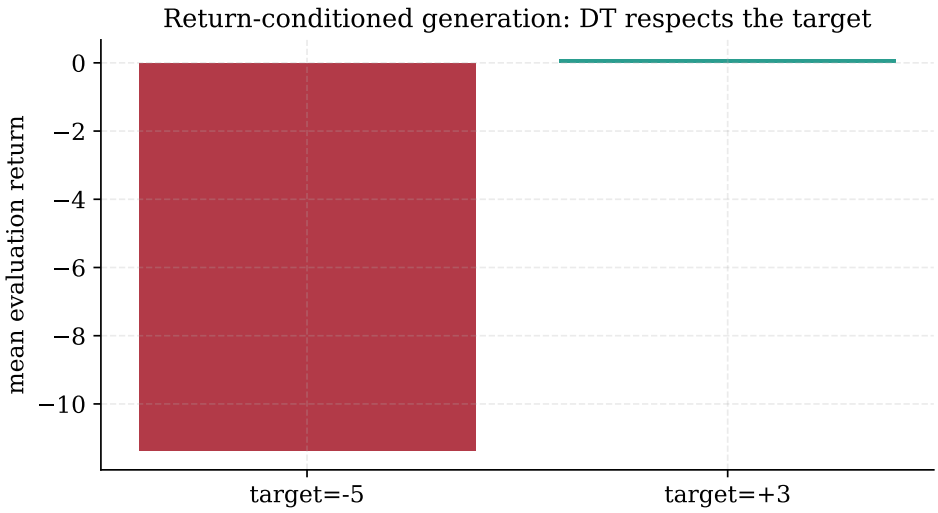


Figure 5.2: Return-conditioned generation: requesting $R = +3$ at the start state yields meaningfully higher episodic returns than requesting $R = -5$. The model has learned to use the return-to-go token as an intent signal.

Safety angle

Sequence-model RL failure modes.

1. **Hallucinated outcomes.** A DT asked for a higher return than ever observed will

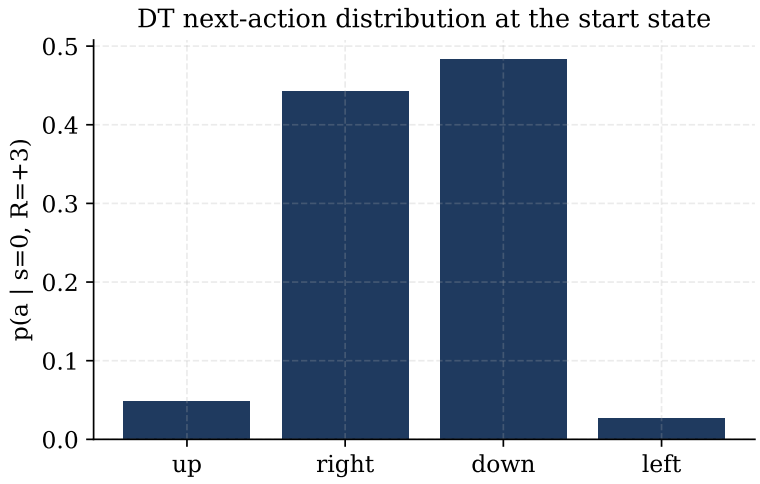


Figure 5.3: DT next-action distribution at $s = 0$ conditioned on $R = +3$. The model places mass on actions consistent with the optimal logger trajectory.

return some action sequence and the user has no signal that the policy is extrapolating. There is no Bellman residual, no value-uncertainty proxy.

2. **Reward hacking by conditioning.** If the dataset contains policies that gamed a proxy metric (e.g. engagement-time as a proxy for satisfaction), DT will replicate the gaming whenever the user supplies a high target.

3. **Mitigations.** (a) Cap the target return at the 95th percentile of observed returns. (b) Pair DT with an FQE estimator; refuse to deploy if the FQE estimate disagrees with the target. (c) Audit generated trajectories for distributional drift from the training set (KL on token frequencies).

Citations: Chen et al. 2021, NeurIPS (DT); Brandfonbrener et al. 2022 "Return-conditioned BC requires unrealistically strong assumptions."

Matching application

DT-style modelling of candidate career trajectories. A candidate's career is a sequence of (*role*, *outcome*, *years*) tuples. Treat *outcome score* as the per-step reward and *eventual career level* as the return-to-go. Train a tiny DT on the historical ATS dataset of completed candidate timelines. Given a current candidate c at stage s_t with a recruiter goal "land role of level R^{target}", the DT outputs the most likely next stage. This is *not* a ranker — it is a generative model of plausible next-step actions for a candidate to take, useful to recruiters as a "what would historically need to happen" tool. The safety-angle considerations apply directly: cap the target outcome, audit for protected-class proxies in the action distribution.

6 Exercises

See `workbook.ipynb`. Highlights:

- **Ex. 1 ($\star$)** Compute returns-to-go via reverse cumsum.
- **Ex. 2 ($\star$)** Construct the causal mask matrix for $L = 3K$ tokens.
- **Ex. 3 ($\star\star$)** Window sampling with left-padding.
- **Ex. 4 ($\star\star$)** Train DT for 600 steps; report final loss.
- **Ex. 5 ($\star\star$)** Verify return-conditioned generation works.
- **Ex. 6 ($\star\star$)** Inspect the action distribution at the start state.
- **Ex. 7 ($\star\star\star$)** Implement the DDPM forward noising step.
- **Ex. 8 ($\star\star\star$, meaningful objective)** Run the career-trajectory demo and discuss the action recommendation.

7 Further reading

See `extended_reading.pdf` for Chen et al. 2021 (DT), Janner et al. 2021 (TT), Janner et al. 2022 (Diffuser), Chi et al. 2023 (Diffusion Policy), and Hansen-Estruch et al. 2023 (IDQL). Sourced from `research/offline_modelbased_sota.md`.